

# COMMONWEALTH OF AUSTRALIA

## Copyright Regulations 1969

### WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Sydney pursuant to Part VB of the Copyright Act 1968 (**the Act**). The material in this communication may be subject to copyright under the Act. Any further copying or communication of this material by you may be the subject of copyright protection under the Act.

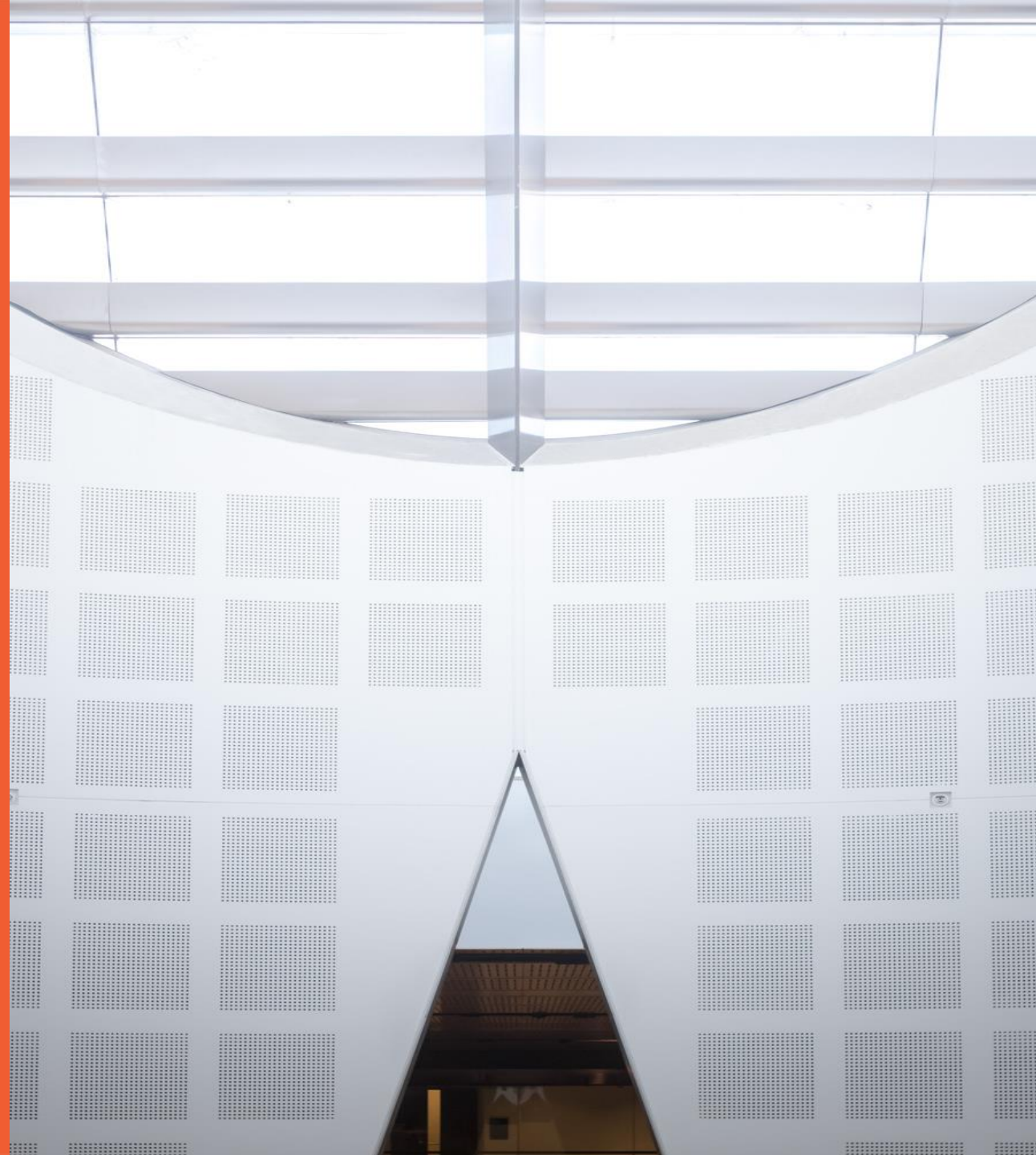
**Do not remove this notice.**

COMPx270: Randomised and  
Advanced Algorithms  
Lecture 6: Hashing and Friends

Clément Canonne  
School of Computer Science



THE UNIVERSITY OF  
SYDNEY



## A question ?

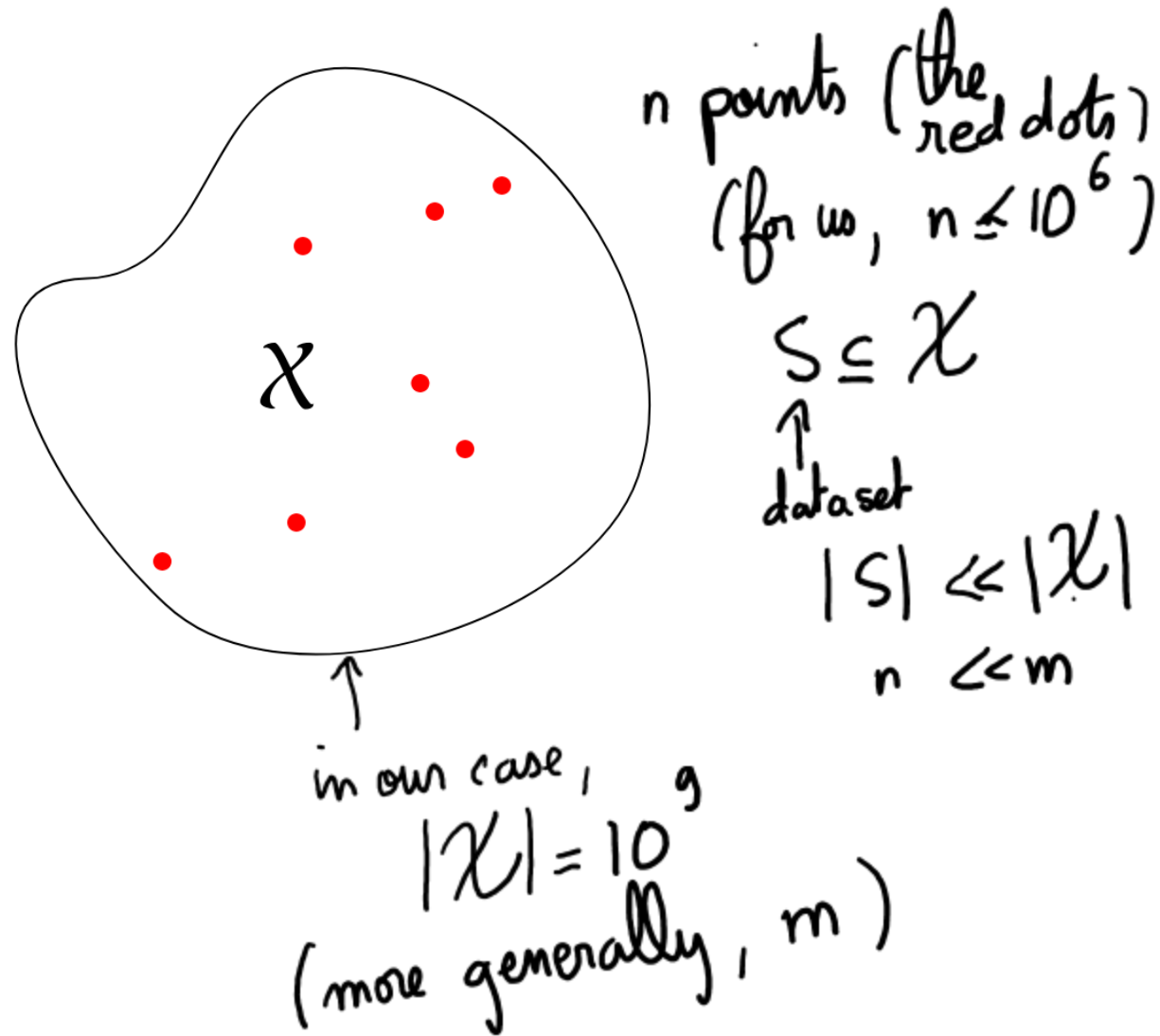
A SID is of the form 450687816 (9 digits, each between 0 and 9). How many distinct SIDs are there?

$$10^9$$

How many distinct students have there even been at U Syd?

much fewer  
 $\approx 10^6$  ish

# Dictionaries (Maps, Associative arrays...)



|        | List                                | Array  | BST*        |
|--------|-------------------------------------|--------|-------------|
| Insert | <del><math>O(1)</math></del> $O(n)$ | $O(1)$ | $O(\log n)$ |
| Lookup | $O(n)$                              | $O(1)$ | $O(\log n)$ |
| Remove | $O(n)$                              | $O(1)$ | $O(\log n)$ |
| Space  | $O(n)$                              | $O(m)$ | $O(n)$      |

## Important detour: data representation

- Indexing something from  $\mathcal{X}$  takes

$$\log_2 |\mathcal{X}| = \log_2 m$$

bits.

We often hide these logs.

$$\mathcal{F} = \{h: \mathcal{X} \rightarrow \mathcal{Y}\}$$

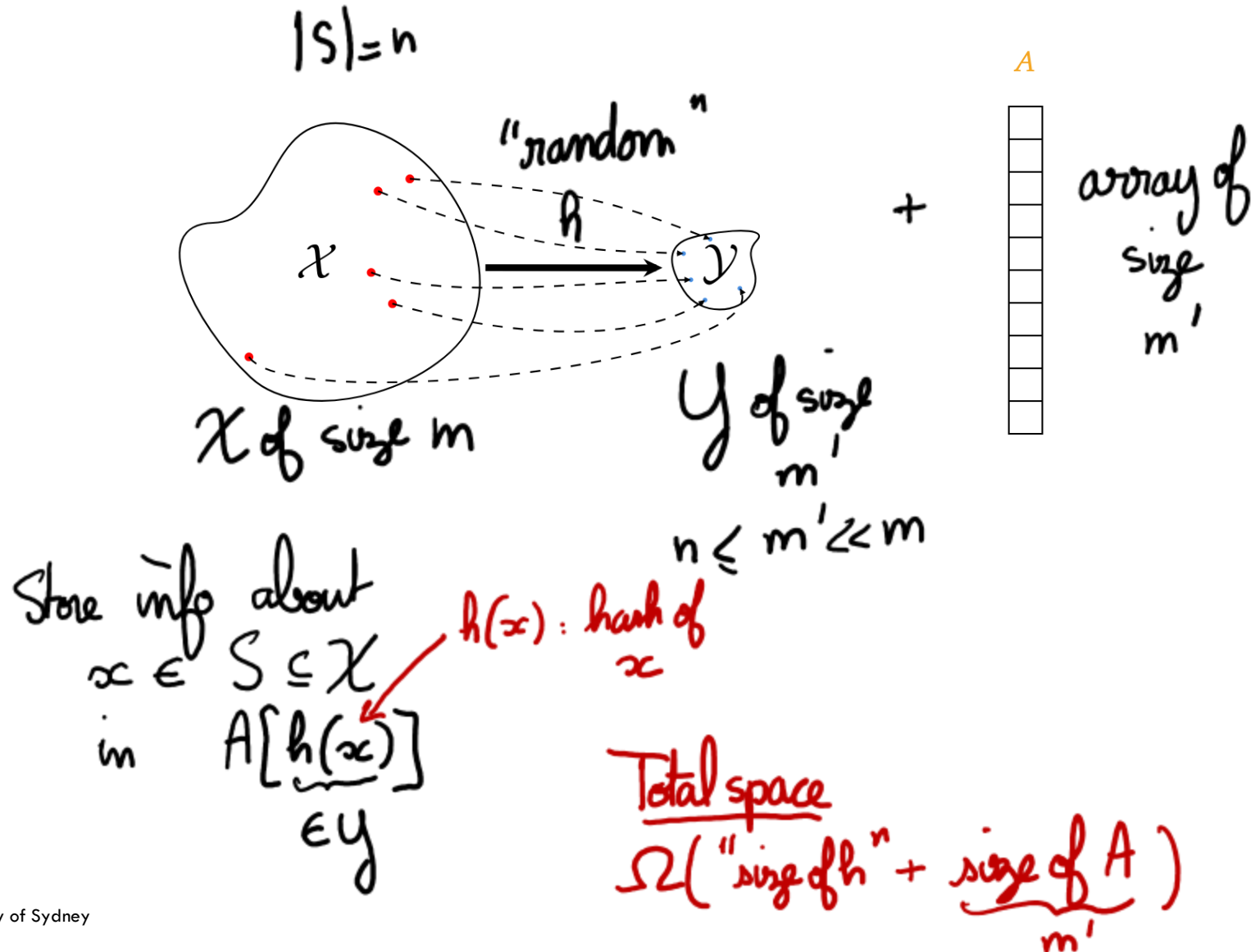
$\uparrow$  size  $m$                        $\uparrow$  size  $m'$

$$|\mathcal{F}| = m'^m$$

Need  $m \log m'$  bits to  
represent an arbitrary function  
from  $\mathcal{X}$  to  $\mathcal{Y}$

# Hash tables

A solution to the Dictionary problem (ADT)



Ideally

time

$O(1)$  insert  
 $O(1)$  lookup  
 $O(1)$  removal

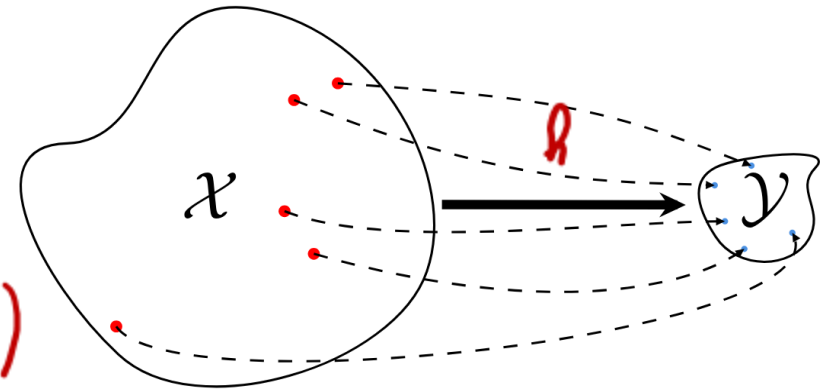
space

$O(n)$

# Hash tables: what is random?

- The subset  $S$  ( $n$  points) is "typical"  
 $\approx$  random

~~No~~ (big assumption)



- The ~~function~~ hash function  $h: X \rightarrow Y$  is random

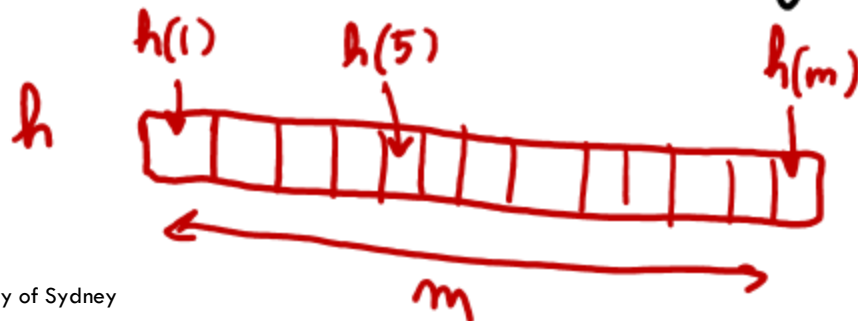
~~No~~ (too much space)

- The hash function  $h$  is "sort of random"

Pick  $h$  u.a.r. from  $\mathcal{H} \subset \mathcal{F} = \{X \rightarrow Y\}$

small  
①

too big



Want ① and ②

②  $\forall x, x' \quad P[h(x) = h(x')] \leq \frac{1}{|Y|}$

# Hash tables: hash families

**Definition 31.1.** A family of functions  $\mathcal{H} \subseteq \{h: \mathcal{X} \rightarrow \mathcal{Y}\}$  is a *universal hash family*, if, for every  $x, x' \in \mathcal{X}$  with  $x \neq x'$ ,

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(x')] \leq \frac{1}{|\mathcal{Y}|}$$

where the probability is over the uniformly random choice of  $h \in \mathcal{H}$ .

# Hash tables: hash families

**Definition 31.1.** A family of functions  $\mathcal{H} \subseteq \{h: \mathcal{X} \rightarrow \mathcal{Y}\}$  is a *universal hash family*, if, for every  $x, x' \in \mathcal{X}$  with  $x \neq x'$ ,

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(x')] \leq \frac{1}{|\mathcal{Y}|}$$

where the probability is over the uniformly random choice of  $h \in \mathcal{H}$ .

**Lemma 31.1.** *Every strongly universal hash family is also a universal hash family. Moreover, there exist universal hash families which are not strongly universal.*

# Hash tables: hash families

**Definition 31.1.** A family of functions  $\mathcal{H} \subseteq \{h: \mathcal{X} \rightarrow \mathcal{Y}\}$  is a *universal hash family*, if, for every  $x, x' \in \mathcal{X}$  with  $x \neq x'$ ,

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(x')] \leq \frac{1}{|\mathcal{Y}|}$$

where the probability is over the uniformly random choice of  $h \in \mathcal{H}$ .

**Theorem 32.** Fix a prime number  $p$ . For given integers  $a, b$ , define the function  $h_{a,b}: \mathbb{Z}_p \rightarrow [m']$  by

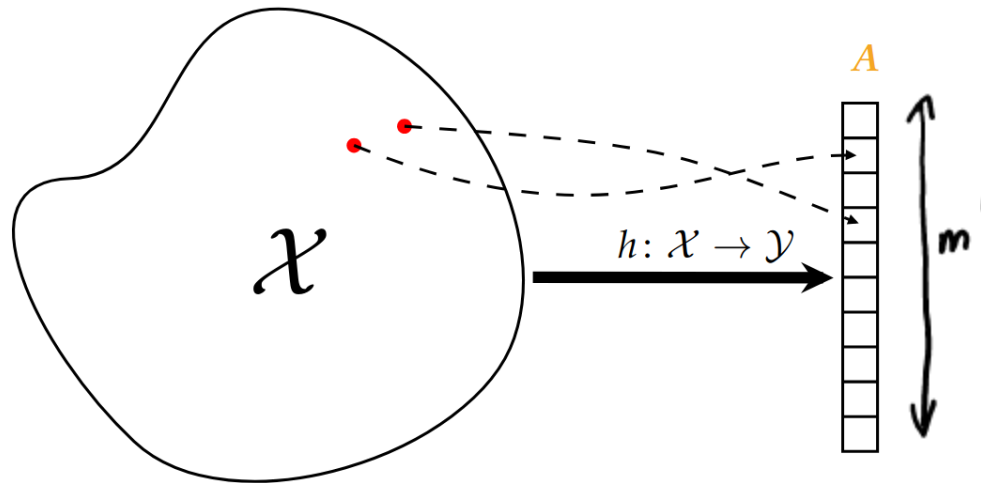
$$h_{a,b}(x) = (ax + b \bmod p) \bmod m', \quad x \in \mathbb{Z}_p$$

and let  $\mathcal{H} = \{h_{a,b}\}_{1 \leq a < p, 0 \leq b < p}$ . Then  $\mathcal{H}$  is a universal hash family of size  $|\mathcal{H}| \leq 2^{2 \log_2 p}$ .

# Hash tables: the data structure

$$H = (\mathcal{X}, n, m, m', \text{"strategy"})$$

Store:  $h, A$



Function  $h$   
chosen once and for  
all **when creating the**  
**data structure**  
 $(h \sim \mathcal{H})$

INSERT( $x$ ):  $A[h(x)] \leftarrow 1$

LOOKUP( $x$ ):  $A[h(x)] \stackrel{?}{=} 1$

REMOVE( $x$ ):  $A[h(x)] \leftarrow 0$

SPACE:  $O(\log |\mathcal{H}| + m')$

Ideally  
(~~cannot~~  
actually  
do that)

Can we set  $m' = O(n)$ ?

Collisions

$x, x' \in S$   
 $x \neq x'$  but  $h(x) = h(x')$

# Hash tables: no collisions? 🍰 ("perfect hashing")

$$\cancel{m' = O(n)}$$

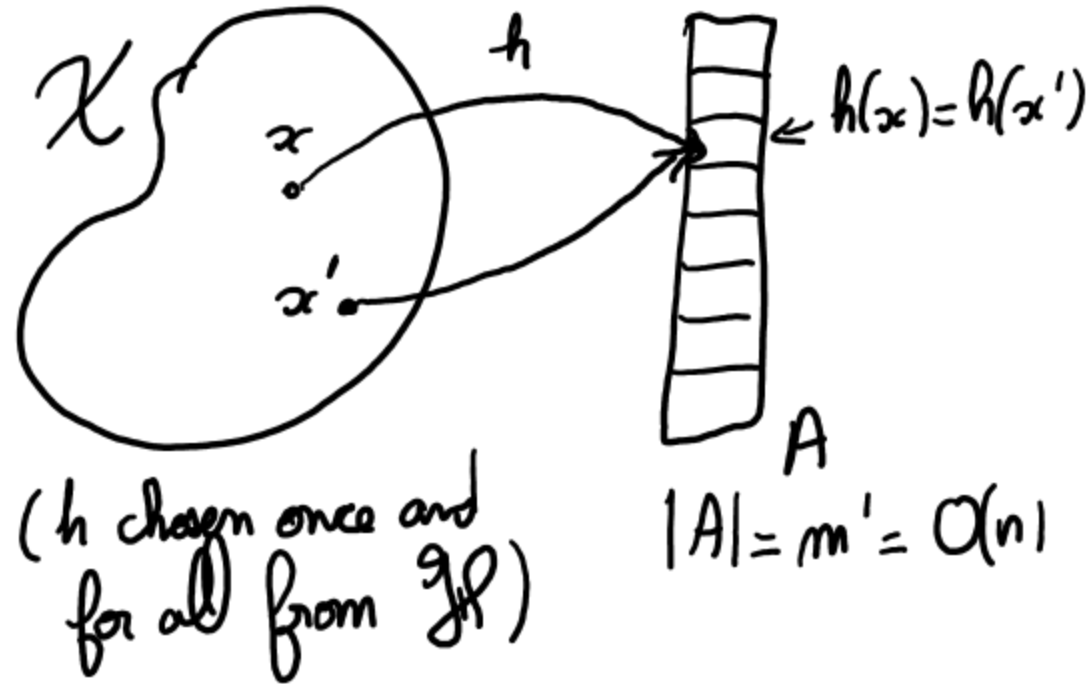
- Birthday paradox:  $m' = \Omega(n^2)$  (usually way too much)
- Worse: if  $m' = O(n)$ , even if  $h$  was chosen truly at random there would be some hash w/

$$O\left(\frac{\log n}{\log \log n}\right)$$

collisions.

Need a strategy to handle collisions.

## Hash tables: ~~no~~ collisions ✨



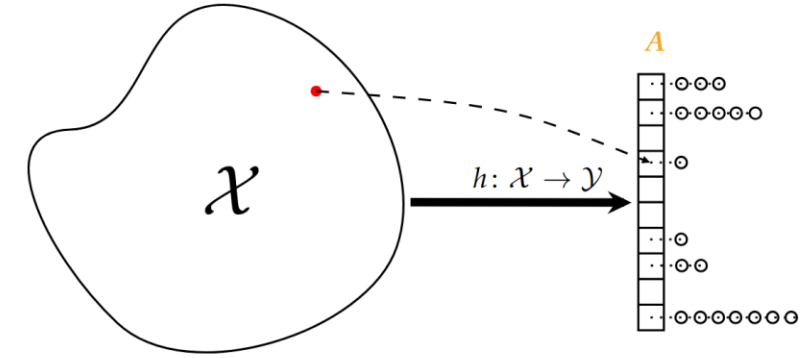
- ① Store  $x$  in  $A[h(x)]$ , not just a bit  
→ allows to detect a collision
- ② Strategies:
  - Chaining
  - Open addressing

# Handling collisions: separate chaining

INSERT( $x$ ): call  $A[h(x)].\text{INSERT}(x)$

LOOKUP( $x$ ): return  $A[h(x)].\text{LOOKUP}(x)$

REMOVE( $x$ ): call  $A[h(x)].\text{REMOVE}(x)$



Load of the hash table

$$\alpha = \frac{n}{m'}$$

Claim. all 3 operations have  
expected time complexity  
 $O(1 + \alpha)$

over the  
initial choice of  $h \sim \mathcal{H}$

Worst-case  $O(n)$

$\alpha > 1$  is possible  
for chaining

# Handling collisions: open addressing

INSERT( $x$ ):

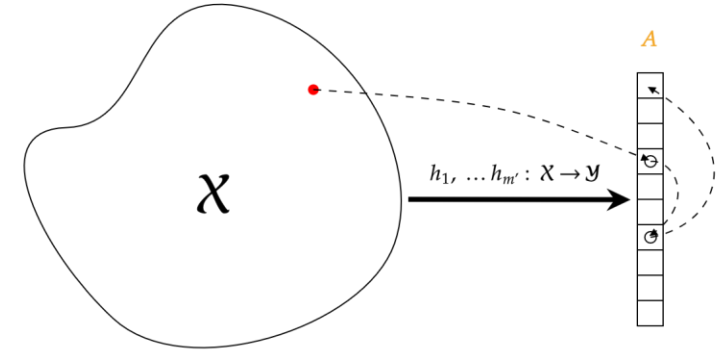
```
for all  $1 \leq t \leq m'$  do
  if  $A[h_t(x)] = x$  then
    return ▷ Already there
  else if  $A[h_t(x)] = \emptyset$  or  $A[h_t(x)] = \perp$  then
     $A[h_t(x)] \leftarrow x$  ▷ Insert it in the first available bucket
  return
```

LOOKUP( $x$ ):

```
for all  $1 \leq t \leq m'$  do
  if  $A[h_t(x)] = x$  then
    return yes
  else if  $A[h_t(x)] = \emptyset$  then
    return no ▷ If  $x$  was present, it'd have been found earlier
```

REMOVE( $x$ ):

```
for all  $1 \leq t \leq m'$  do
  if  $A[h_t(x)] = x$  then
     $A[h_t(x)] \leftarrow \perp$  ▷ Special symbol to indicate there was something before
  else if  $A[h_t(x)] = \emptyset$  then
    return ▷ If  $x$  was present, it'd have been found earlier
```



Need  $\alpha \leq 1$  for open addressing

Assumption: we have  $h_1, \dots, h_{m'}$  "suitable" hash functions.

$\forall x, h_1(x), \dots, h_{m'}(x)$  "covers" all  $m'$  possible cells in some order.

# Handling collisions: open addressing

**Theorem.** Assuming **stuff** ( $\dagger$ ), the expected time complexity of LOOKUP is

$$O\left(\frac{1}{1-\alpha}\right).$$

**stuff:**  $\forall x, (h_1(x), h_2(x), \dots, h_{m'}(x))$  is a u.a.r. permutation of  $\{1, \dots, m'\}$

**Proof:**

$$\mathbb{E}[T(n, m')] = O(1) + \alpha \cdot \mathbb{E}[T(n-1, m'-1)]$$

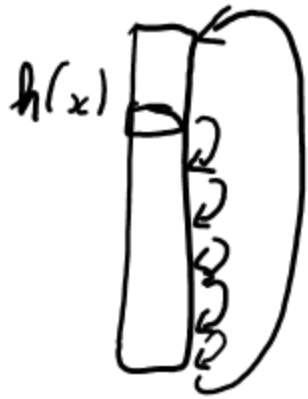
time for a lookup on a query  $x$  in a hash table with  $n$  points and size  $m'$   $\leq$   $O(1) + \alpha \mathbb{E}[T(n, m')]$  (handwaving)

$$\rightarrow \mathbb{E}[T(n, m')] \leq \frac{O(1)}{1-\alpha}$$

□

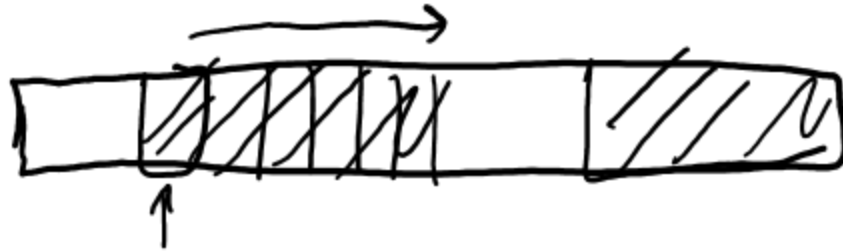
## Handling collisions: open addressing (linear probing)

~~$h_1, \dots, h_{m'}$~~   $\rightarrow$  only one  $h$



$$\begin{aligned} h_1(x) &= h(x) & [m'] \\ h_2(x) &= h(x) + 1 & [m'] \\ h_3(x) &= h(x) + 2 & [m'] \\ & \vdots & \\ h_{m'}(x) &= h(x) + m' - 1 & [m'] \end{aligned}$$

# Handling collisions: open addressing (linear probing)



**Theorem (Knuth'62).** Under some **reasonable** assumption, the expected time complexity of LOOKUP is

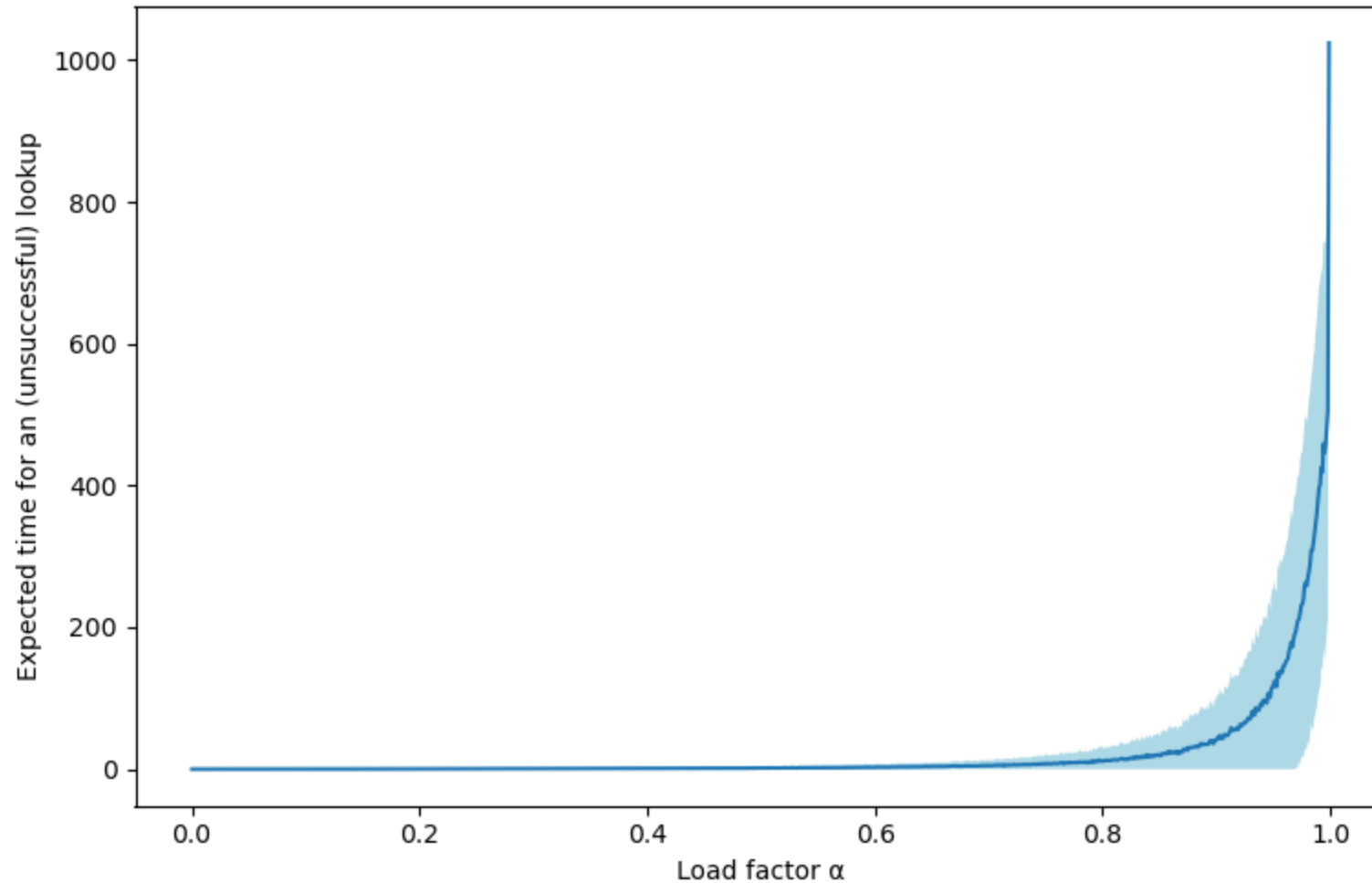
<sup>^</sup>  
unsuccessful

$$O\left(\frac{1}{(1 - \alpha)^2}\right).$$

# Handling collisions: open addressing (linear probing)

**Theorem (Knuth'62).** Under some **reasonable** assumption, the expected time complexity of LOOKUP is

$$O\left(\frac{1}{(1-\alpha)^2}\right).$$



# Handling collisions: open addressing (cuckoo hashing)

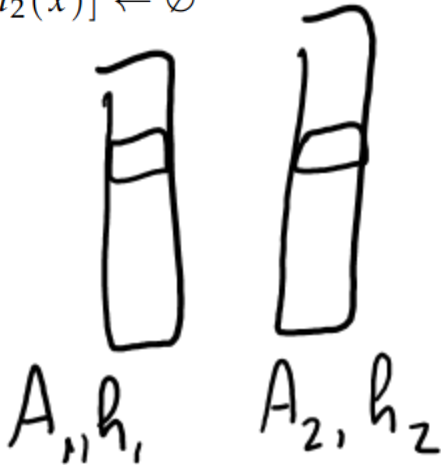


LOOKUP( $x$ ):

```
if  $A_1[h_1(x)] = x$  or  $A_2[h_2(x)] = x$  then  
  return yes  
else  
  return no
```

REMOVE( $x$ ):

```
if  $A_1[h_1(x)] = x$  then  
   $A_1[h_1(x)] \leftarrow \emptyset$   
else if  $A_2[h_2(x)] = x$  then  
   $A_2[h_2(x)] \leftarrow \emptyset$ 
```



→  $O(1)$  time!



INSERT

# Handling collisions: open addressing (cuckoo hashing)



INSERT( $x$ ):

if  $A_1[h_1(x)] = \emptyset$  then

$A_1[h_1(x)] \leftarrow x$

else if  $A_2[h_2(x)] = \emptyset$  then

$A_2[h_2(x)] \leftarrow x$

else

$T \leftarrow 0$

▷ The evictions start

$x' \leftarrow A_1[h_1(x)]$

$A_1[h_1(x)] \leftarrow x$

while  $T < T_{\max}$  do

$x'$  goes to  $A_2[h_2(x')]$ , and if there was something there, that element now needs to move, etc.

$T \leftarrow T + 1$



# Handling collisions: open addressing (cuckoo hashing)



**Theorem (Pagh–Flemming'01).** Cuckoo hashing achieves worst-case time  $O(1)$  for LOOKUP and REMOVE, and **expected** time  $O(1)$  for INSERT.

# Hash tables: summary

- The hash function  $h$  is **not** a "truly random function", but is chosen **once and for all**, at random  , from a **small** hash family  $\mathcal{H}$
- $h$  is then kept as part of the data structure:  $O(\log |\mathcal{H}|)$  bits
- The **data** itself is **not** assumed to be random: only the choice of  $h$  from  $\mathcal{H}$  is
- **No silver bullet**: things are only good in expectation (or with high probability)

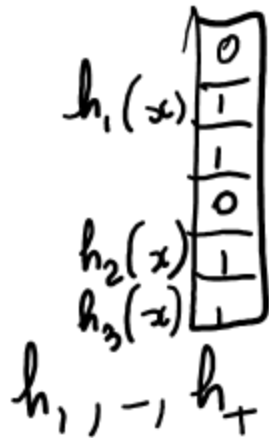
# Can we do better? Bloom filters!

Faster, less space, **but**... LOOKUP is sometimes **wrong**

Faster:  $O(1)$  time complexity in the worst case (for read!)

Less space (by constant factor)

But LOOKUP sometimes wrong: false positive



$$\text{LOOKUP}(x) : A[h_1(x)] \wedge A[h_2(x)] \wedge \dots \wedge A[h_t(x)]$$

# Perspective: why does it work so well? 🙋

"Hashing is fundamental to many algorithms and data structures widely used in practice. For theoretical analysis of hashing, there have been two main approaches. First, one can **assume that the hash function is truly random**, mapping each data item independently and uniformly to the range. **This idealized model is unrealistic** because a truly random hash function requires an exponential number of bits to describe. Alternatively, one can provide rigorous bounds on performance when **explicit families of hash functions are used, such as 2-universal or  $O(1)$ -wise independent families**. For such families, performance guarantees are often noticeably weaker than for ideal hashing.

In practice, however, it is commonly observed that simple hash functions, including 2-universal hash functions, perform as predicted by the idealized analysis for truly random hash functions."

📄 *"Why Simple Hash Functions Work: Exploiting the Entropy in a Data Stream."*  
Mitzenmacher and Vadhan, 2008